

ALLIANCE UNIVERSITY

Entity Relationship Diagram & Relational Schema

Domain: Cab Booking System

Name:	Razin Basha R
Roll No:	2511021240012
Course:	Database Management System
Programme:	BCA AI/ML
Semester:	IIIrd

1. Entities and Attributes

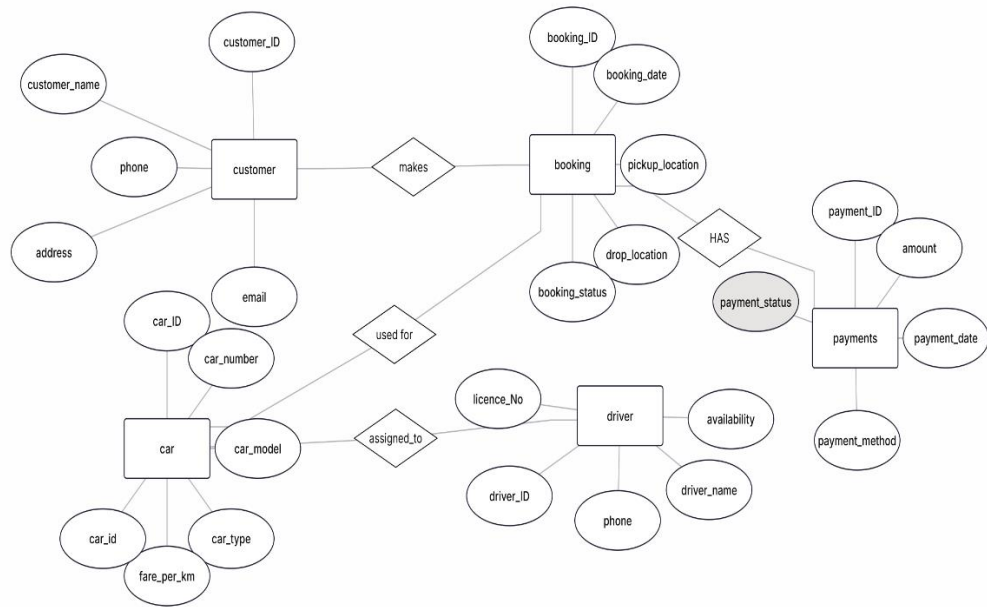
The Cab Booking System consists of five core entities, carried over from the First DBMS Assignment. Primary Key (PK) attributes are shown in bold.

Entity	Primary Key	Other Attributes
CUSTOMER	Customer_ID (PK)	Customer_Name, Phone, Email, Address
DRIVER	Driver_ID (PK)	Driver_Name, Phone, License_No, Availability
CAR	Car_ID (PK)	Car_Number, Car_Model, Car_Type, Fare_Per_KM
BOOKING	Booking_ID (PK)	Booking_Date, Pickup_Location, Drop_Location, Distance_KM, Booking_Status
PAYMENT	Payment_ID (PK)	Amount, Payment_Date, Payment_Method, Payment_Status

2. Relationships and Cardinality

Relationship	Cardinality	Description
CUSTOMER — MAKES — BOOKING	One-to-Many (1:N)	A customer can make many bookings over time, but each booking is made by exactly one customer.
DRIVER — ASSIGNED TO — CAR	One-to-Many (1:N)	A driver can be assigned to many cars, but each car is assigned to exactly one driver at a time.
CAR — USED FOR — BOOKING	One-to-Many (1:N)	A car can be used for many bookings over time, but each booking uses exactly one car.
BOOKING — HAS — PAYMENT	One-to-One (1:1)	Each booking generates exactly one payment record, and each payment is linked to exactly one booking.

3. ER Diagram



4. Relational Schema

The ER Diagram is converted into the following relational schema. Each entity becomes a table, and every 1:1 or 1:N relationship is implemented by placing a foreign key on the 'many' side (or on either side for 1:1). Since this design contains no Many-to-Many relationships, no additional associative/junction tables are required.

CUSTOMER

- **Customer_ID (PK)**
- Customer_Name
- Phone
- Email
- Address

DRIVER

- **Driver_ID (PK)**
- Driver_Name
- Phone
- License_No
- Availability

CAR

- **Car_ID (PK)**
- Car_Number
- Car_Model
- Car_Type
- Fare_Per_KM
- Driver_ID (FK → DRIVER)

BOOKING

- **Booking_ID (PK)**
- Customer_ID (FK → CUSTOMER)
- Car_ID (FK → CAR)
- Booking_Date
- Pickup_Location
- Drop_Location
- Distance_KM
- Booking_Status

PAYMENT

- **Payment_ID (PK)**
- Booking_ID (FK → BOOKING)
- Amount
- Payment_Date
- Payment_Method

- Payment_Status

5. Relationship Explanation

CUSTOMER – MAKES – BOOKING (1:N)

A single customer can make many bookings over time (different trips on different dates), but each individual booking is made by exactly one customer. This one-to-many relationship is implemented by storing Customer_ID as a foreign key in the BOOKING table.

DRIVER – ASSIGNED TO – CAR (1:N)

A driver can be assigned to multiple cars over time, but at any given time each car is assigned to exactly one driver. This one-to-many relationship is captured by placing Driver_ID as a foreign key in the CAR table.

CAR – USED FOR – BOOKING (1:N)

A single car can be used to fulfil many bookings over its lifetime, but each individual booking uses exactly one car for that trip. This one-to-many relationship is implemented by storing Car_ID as a foreign key in the BOOKING table.

BOOKING – HAS – PAYMENT (1:1)

Each booking results in exactly one corresponding payment record, and each payment is tied to exactly one booking. This one-to-one relationship is implemented by storing Booking_ID as a foreign key in the PAYMENT table.